



サイボウズにおけるGitHubの 全社的な利用改善、セキュリティ、 ガバナンス強化の取り組み

GitHub OctoNihon

2026年9月17日

発表者: サイボウズ 生産性向上チーム 河野 雄也

自己紹介

河野 雄也

サイボウズ株式会社 生産性向上チーム

サイボウズでソフトウェアエンジニアとして勤務。社内向けにGitHub Dependency Graphを使ったサプライチェーン攻撃の影響チェッカーや、AWS SecurityHubの通知システムを開発するなど、GitHub・AWSの活用促進に取り組んでいる。

目次

01	サイボウズはグループウェアを作っています	P04	12	安全な方が使いにくかった	P15
02	サイボウズの開発組織の構造と規模	P05	13	Fine-grained PATの承認を不要にするまで	P16
03	生産性向上チームについて	P06	14	ルールの改定を起案して検査する	P17
04	社内でのGitHub活用にはこんな要望があった	P07	15	安全な利用方法を社内ルールへ反映する	P18
05	侵害の影響範囲を調べる	P08	16	ルールの制定に合わせて検査するプログラムを書く	P19
06	侵害されたパッケージの影響範囲をSBOMで調べる	P09	17	ツールを拡張する	P20
07	SBOMを検索するツールを作って社内に展開した	P10	18	増え続けるツールをどう拡張していくか	P21
08	利用者を把握する	P11	19	検査はどれも同じ形をしている	P22
09	GitHubアカウントと社員情報を結びつける	P12	20	同じ枠組みで足していった検査	P23
10	安全なトークンへ寄せる	P13	21	実行した人にしか分からない状態を変える	P24
11	安全な設定へ変えるのに時間がかかった	P14	22	開発チームとしてどう向き合ったか	P25

サイボウズはグループウェアを作っています



ノーコードで業務アプリを作れるクラウドサービス



大規模な組織向けのグループウェア



中小企業向けのグループウェア



チームでのメール対応を一元管理するメール共有システム

これら載せるクラウド基盤 cybozu.com も、自社で開発して運用しています

サイボウズの開発組織の構造と規模

製品を作る組織と、クラウド基盤を運用する組織が、本部として分かれています。複数の開発組織を技術面で統括する役職や部署は置いていません。

組織

- **開発本部** — 製品を開発する
- **クラウド基盤本部** — cybozu.comを運用する
- **情シス** — 社内のIT部門で、GitHubなどを管理する

サイボウズのGitHub

- 29のOrganization
- 700以上のアカウント
- 管理者はOrganizationごとに違う

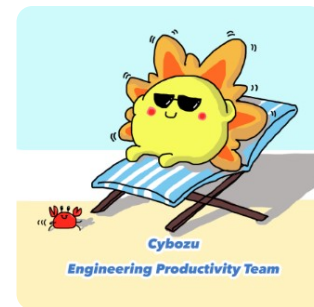
全体に影響する変更は、開発本部長とクラウド基盤本部長の両方の承認が要る

生産性向上チームについて

サイボウズの生産性向上チームは、開発者が使うシステムを社内に提供しています。この発表では、サイボウズのGitHub Enterprise Cloud（GHEC）で見つけた課題に、どのような形で答えてきたかを話します。

提供しているシステム

- AWSのアカウント管理
- 開発用のSSLリバースプロキシ
- GitHub Actionsの
 - セルフホストランナー
 - Claude Code Action（クォータでコストを制御）



社内でのGitHub活用にはこんな要望があった

安全に使いたい

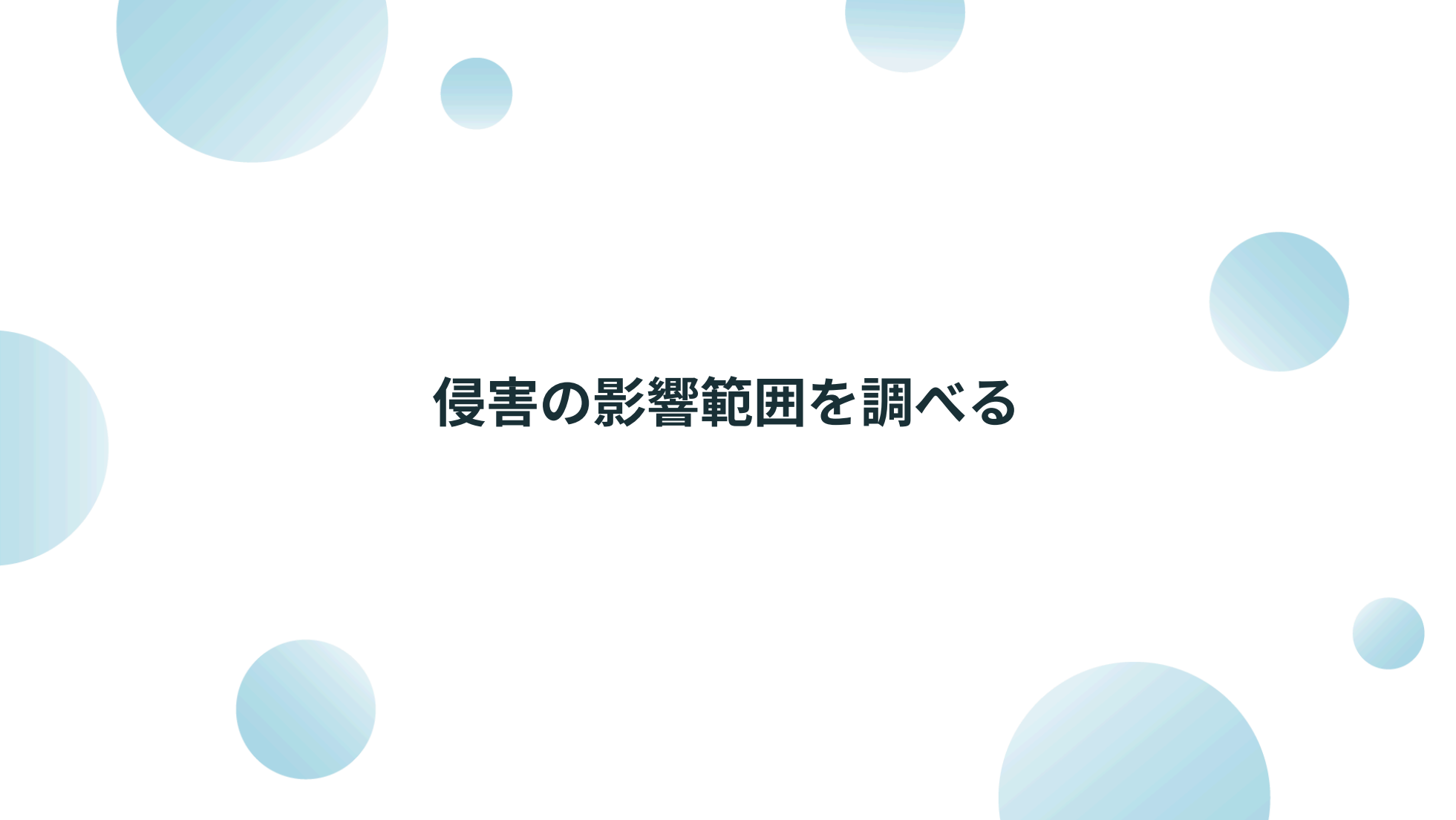
- 侵害が報じられても、影響範囲をすぐ調べられない
- 他人が書いたActionをそのまま信用してよいかわからない

管理の手間を減らしたい

- アカウントが社内の誰に対応しているかわからない

もっと使いやすくしたい

- 安全な手段は承認が必要などで扱いにくく、危険な手段に流れてしまう



侵害の影響範囲を調べる

侵害されたパッケージの影響範囲をSBOMで調べる

ソフトウェアサプライチェーン攻撃は、2025年ごろから活発になりました。依存パッケージなど信頼された経路を通じて侵害コードを送り込む攻撃です。2025年9月に報じられたnpmの事例では、200件近いパッケージに影響が出ました。

これまでの調べ方

- 侵害されたパッケージを手作業で検索するか、検証スクリプトを都度書いていた
- 検証方法がチームごとに違い、取りこぼしが起きる

SBOM（使うパッケージの一覧）で調べる

- Dependency Graphが生成したものを `GET /repos/{owner}/{repo}/dependency-graph/sbom` で取得できる
- パッケージ名で引けば、確認が必要なりポジトリを絞り込める
- 対象はデフォルトブランチだけ

SBOMを検索するツールを作って社内に展開した

sbom-checker

- 指定したパッケージを使っているリポジトリをSBOMから探すCLI
- 手作業の一次調査を置き換えるために作り、社内へ配布した

1. 調べたいパッケージ名をCSVで渡す
2. Organization内の全リポジトリのSBOMを取得して照合する
3. 検出したりポジトリとバージョンを出力する

```
SBOM パッケージチェックツール
リストファイル: list.csv

読み込み完了: 10 個のパッケージ

=== 検出されたパッケージ ===

[repo-a] example-logger (example-logger@1.2.3)

=== サマリー ===
チェックしたSBOM数: 10
監視対象パッケージ数: 10
検出されたパッケージ数: 1
```

複数リポジトリの一次調査を、同じ手順で再実行できるようにした

The background of the slide is white and features several semi-transparent blue circles of various sizes scattered across the frame. The circles have a subtle gradient, giving them a three-dimensional appearance.

利用者を把握する

GitHubアカウントと社員情報を結びつける

このGitHubアカウントは、社内の誰なのか

社員は個人のメールアドレスで個人アカウントを作る。社内システムとの対応が取れない。

- `GET /enterprises/{enterprise}/consumed-licenses` で利用者一覧を取得する
- シングルサインオンのメールアドレスで社員情報と突き合わせる
- 所属OrganizationとOwner権限を社員ごとに記録する

社員とGitHubアカウントの対応は、棚卸しやアクセス権の調査だけでなく、各所へ連絡するときの名簿としても使える

GitHubユーザー対応表

GitHubログイン	社員	メールアドレス	所属組織	所属GitHub Org	オーナーOrg
alice-gh	アリス	alice@example.com	第1開発部	example-dev, example-infra	example-dev
bob99	ボブ	bob@example.com	品質保証部	example-qa	-
charlie-oss	チャーリー	charlie@example.com	基盤運用部	example-infra, example-dev	example-infra



安全なトークンへ寄せる

安全な設定へ変えるのに時間がかかった

- 社内ルール上、GHECは**開発本部長とクラウド基盤本部長**が責任を持つ
- 日々GHECを管理しているのは情シスだが、判断はこの2名に任されてきた
- 全メンバーが所属するデフォルトのOrganizationは、設定を1つ変えるにも両名の判断が要った
- チーム独自に管理者を置くOrganizationもあった

責任の所在は決まっていたが判断の委譲が追いつかず、決定が四半期単位でずれ込みやすかった

安全な方が使いにくかった

- Classic PATは、かなり広い権限を持つトークン
 - サイボウズでは承認不要で作成できる
- Fine-grained PATは、権限と対象リポジトリを絞って発行できるトークン
 - サイボウズではOrganization ownerの承認が必要
- セキュリティ上の懸念から、可能な限りFine-grained PATへ寄せたかった

PATをめぐる課題

作りにくさが、Classic PATを選ぶ理由の1つになっていた

Fine-grained PATの承認を不要にするまで

生産性向上チーム

課題と影響を調査し、変更案を提案する

開発本部長とクラウド基盤本部長

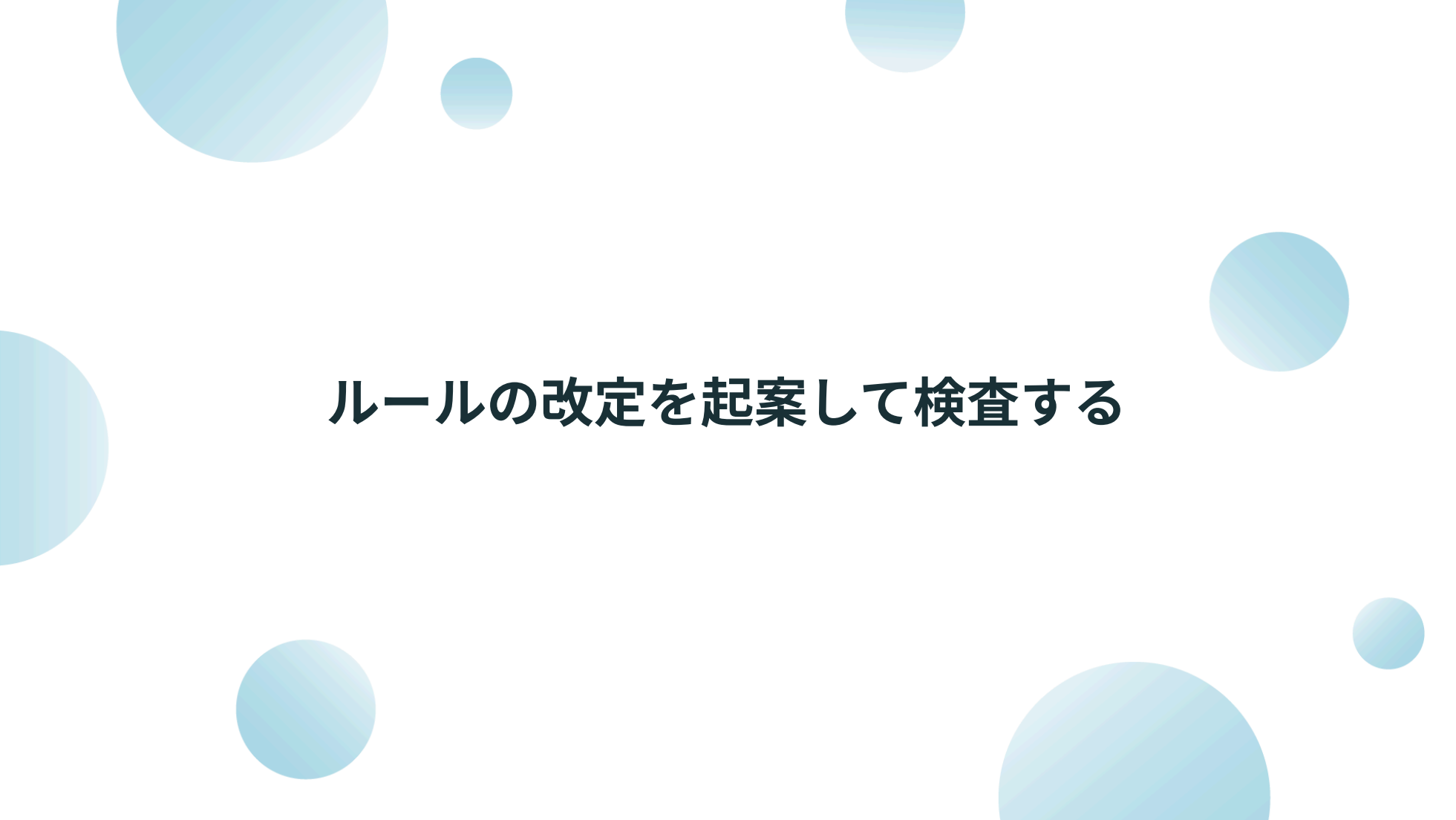
全体へ影響する方針を承認する

情シス

管理権限を使い、設定を変更する

- 承認を外すと、ownerが発行時に権限と対象リポジトリを点検する機会は失われる
- Classic PATを許している以上、Fine-grained PATでリスクは増えない
- Classic PATはGitHub Packagesなどで必要な場面が残るため、いまは限定的に使う

社内のセキュリティに関するWGとも議論し、承認は不要になった



ルールの改定を起案して検査する

安全な利用方法を社内ルールへ反映する

改定案を自分たちで起案し、社内のセキュリティに関するWGへ提案しています。これらは社内の様々なチームと分担して進めています。

公開直後の更新は取り込まない

依存更新をリリースから14日間は取り込まない。
セキュリティ更新は例外として扱う

Actionをcommit SHAで固定する

GitHub Actionsの `uses:` の参照を、タグではなくcommit SHAで固定する

Actionsのデフォルト権限を絞る

Enterprise配下の全Organizationで、
`GITHUB_TOKEN` の既定を読み取りのみにする

ルールの制定に合わせて検査するプログラムを書く

ルールを決めても、各リポジトリが従っているかは分かりません。設定ファイルを1つずつ開いて回るのも現実的ではありません。そこで、ルールの制定に合わせて、準拠を検査するプログラムを書いています。

例 release-age-scan

- 依存更新をリリースから14日間は取り込まないというルールを検査する
- セキュリティチームと協力して用意し、社内ルールへ参考情報として掲載

1. Organization配下の全リポジトリから設定ファイルを取得する
2. Renovateの `minimumReleaseAge` とDependabotの `cooldown` を解釈し、実効値を求める
3. 設定を推奨値と比べ、満たさないリポジトリを報告する



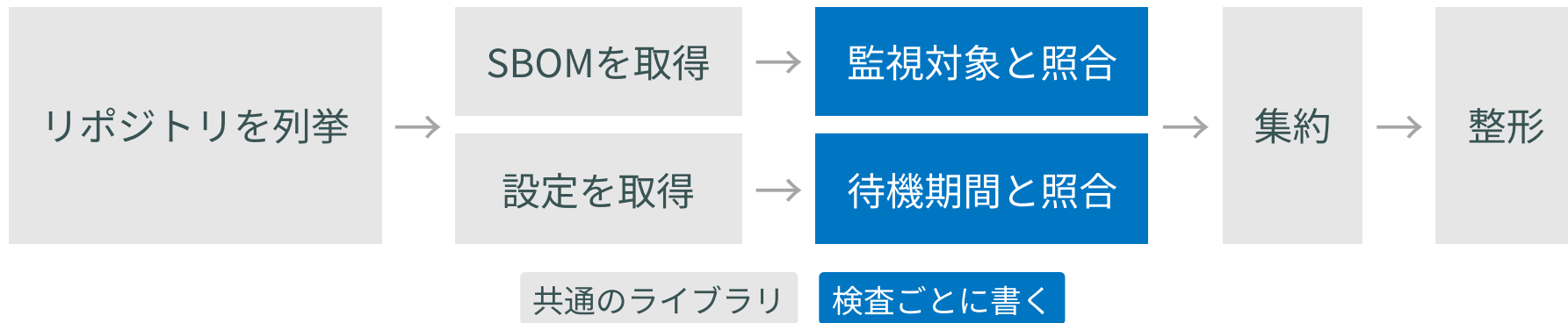
ツールを拡張する

増え続けるツールをどう拡張していくか

- 課題を解決するたびに、`sbom-checker` や `release-age-scan` のような個別のツールを作ってきた
- 改定案が通るたびに、準拠を確かめる検査が新しく要る
- 別々に作り続けると、作る手間もメンテナンスコストも上がっていく

検査はどれも同じ形をしている

リポジトリを列挙し、APIを呼んで検証すべきデータを洗い出し、結果を集約して出す。そこを**ストリームベースのアーキテクチャ**として共通化しました。



- 単一のバイナリにまとめ、社内向けの `gh extension` として配る

同じ枠組みで足していった検査

credential-audit

- `GET /orgs/{org}/credential-authorizations` でシングルサインオンの認可済みトークンを取得し、`admin:org` のような危険な権限を持つものを検出する
- 対象はClassic PATとOAuth app token

unpinned-actions

- `GET /repos/{owner}/{repo}/git/trees/{sha}?recursive=1` でworkflowを探し、SHA固定でない `uses:` 参照を検出する
- ローカル参照などを除外できる

周知と個別確認に頼らずに済むようになり、分かった利用状況が次のルールを起案する材料になる

実行した人にしか分からない状態を変える

- いまは使う人が明示的に実行するCLIである
 - 実行した人にしか、実行したときだけ分かる
- 週次で自動的に回し、結果を集めて見られるようにしようとしている
- そうなると、検査を実装することがルールの適用を進めることにつながる

検査を足せるようにしたので、次は足した検査が効いているかを横断して測る

開発チームとしてどう向き合ったか

- 生産性向上チームは開発チームであり、運用チームでもガバナンスチームでもない
- その立場で全社的な利用改善を進めるために、要望に応じて道具を作ってきた
- その中で、社内ルールを検査するプログラムを作ることを一つのコンセプトにした
- ルールの起案とツールの開発を両輪で回すことで、定めたルールに準拠しているかを確認される

組織と目的に沿った解決策が見えたことで、生産性向上チームによるGitHub改善は軌道に乗りました

